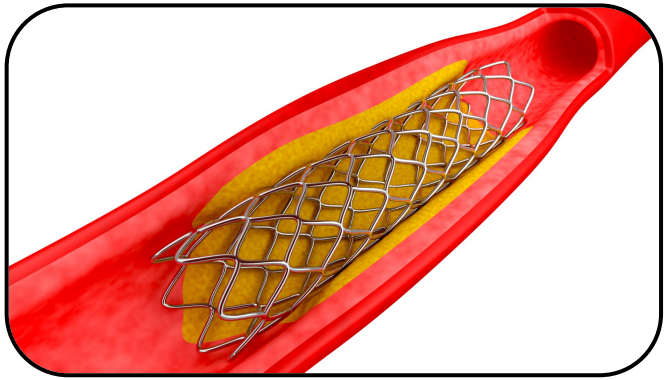
Carnegie Mellon University

Using Computer Vision to Screen Angiograms

Team Denver: Vrishabh Kenkre, Rajiv Joshi, Sangram Sahoo, Shweta Iyer

Heart related disease remains the leading cause of death in the United States



- Stents are used to widen vessel and prevent future attacks

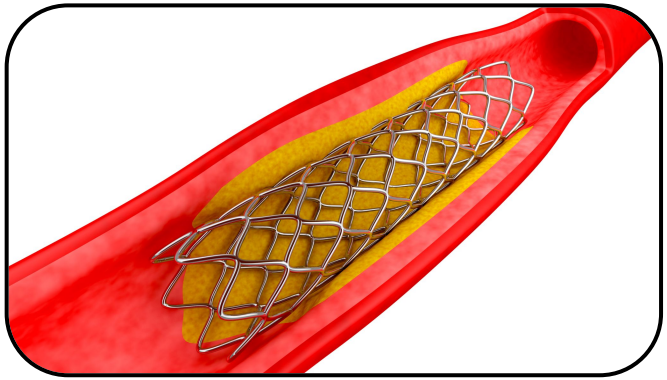
Heart related disease remains the leading cause of death in the United States

CDC 2024: 1 person passes every 34 seconds from a cardiovascular disease → 900,000 deaths in a calendar year US

What people usually do for addressing cardiac issues:

- Meet with cardiologist specialist → procedure checkup for identifying heart conditions→ Stents for any blockages or vessel size issues

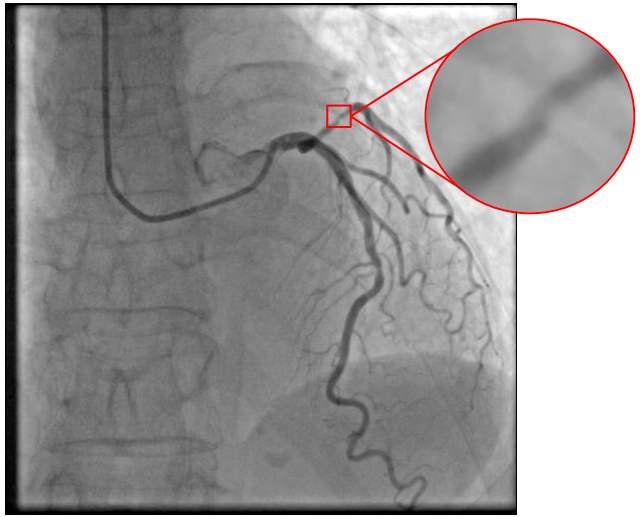
Question: How and what tools do doctors use to identify major heart conditions and how can we leverage Computer vision to enhance this identification for doctors? → Hence we looked into Angiograms



- Stents are used to widen vessel and prevent future attacks

What are coronary angiograms?

- Detect blockages in arteries & veins
- Inject dye into bloodstream using catheter & take X-ray
- Highlights blockages or narrowed arteries
- Doctors can prescribe angioplasty or bypass surgery to restore blood flow



Current methods to identify diseases

- Cardiologists currently identify coronary disease by visually estimating vessel narrowing in angiograms.

- Manual visual assessment → subjective + time-consuming
  -
    - Primarily rely on a doctors ability to identify transitions in blood flow, thinning vessels, and diseases regions

- Decisions can vary doctor to doctor

Cardiologist Visual Inspection

Our proposal: CV for screening angiograms

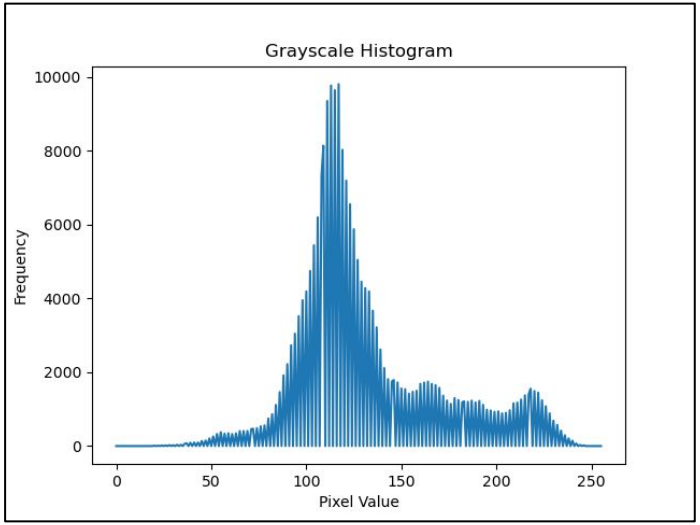
- Pseudocolor-Based Blockage Detection
- Enhanced Binarization Pipeline
- Skeletonization for Diameter & Stenosis Measurement

Integrated automated screening pipeline

- Combine pseudocolor, contour extraction, binarization, and skeletonization.
- Fully transforms raw angiograms into analyzable vessel maps.
- Produces stenosis heatmaps, diameter curves, and localized narrowing markers.
- Supports faster, more objective preliminary screening for cardiologists.

Method 1: Detecting blockages using pseudocoloring

Original



Histogram

Methods for image processing Method 1:

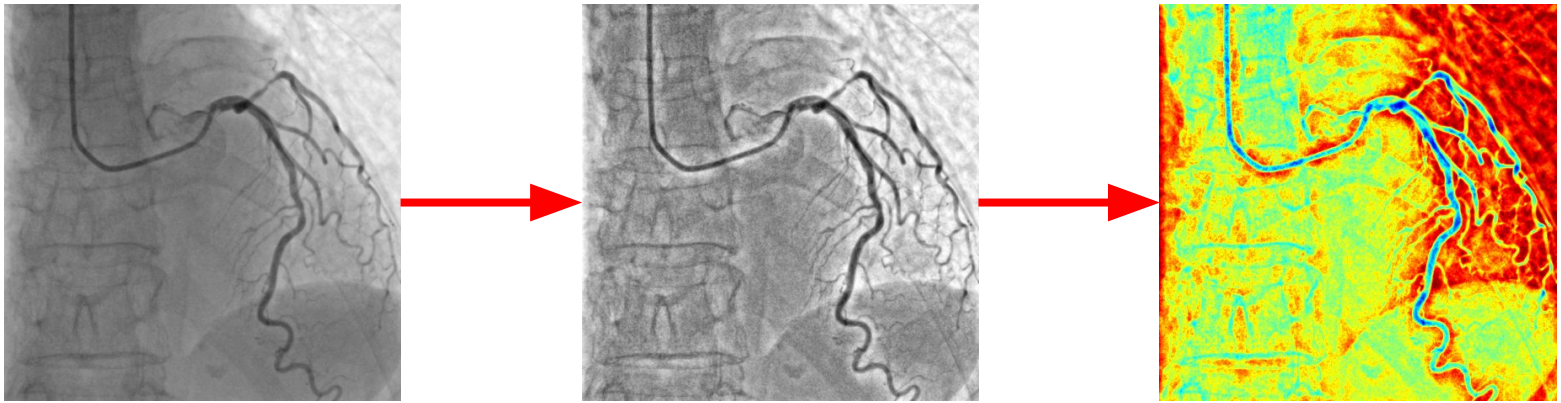
Many different image processing techniques were attempted to enhance the vessels in the given angiograms along with improving the detectable transition of clogged flow regions that show up as dark → light → dark transitions in the angiogram.

For this methods like Otsu and Direct Histogram Equalization failed to enhance the image or separate out the vessels as the Unimodal histogram distribution causes Otsu to fail and the smooth grayscale levels of the images causes histogram equalization to make the vessel disappear or the flow inside a vessel to be a singular grayscale with no transitions in thinned vessels.

Hence we looked into CLAHE and convertScaleAbs().

Pre-processing methods: CLAHE

```
clahe = cv.createCLAHE(clipLimit=4.0, tileGridSize=(30, 30))
enhanced_image = clahe.apply(grayscale_image)
```



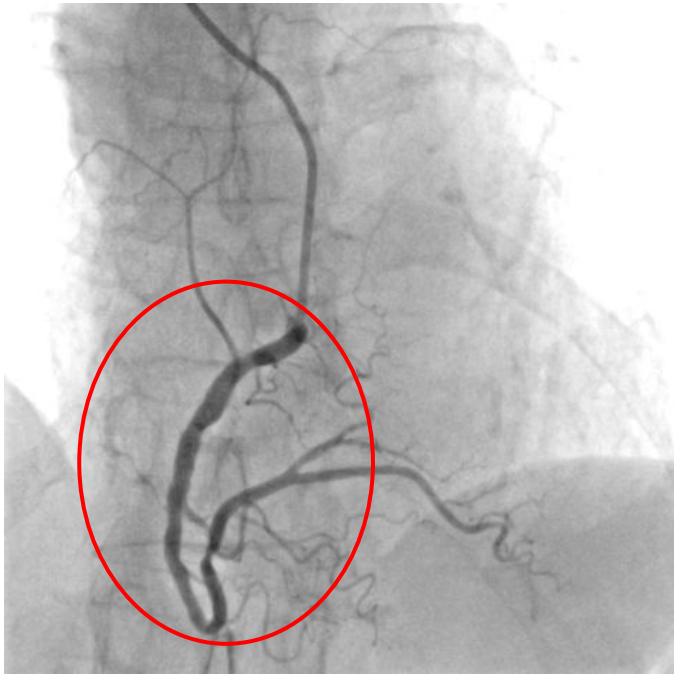
Original CLAHE Pseudocolor

Pre-processing methods: convertScaleAbs and Darkening

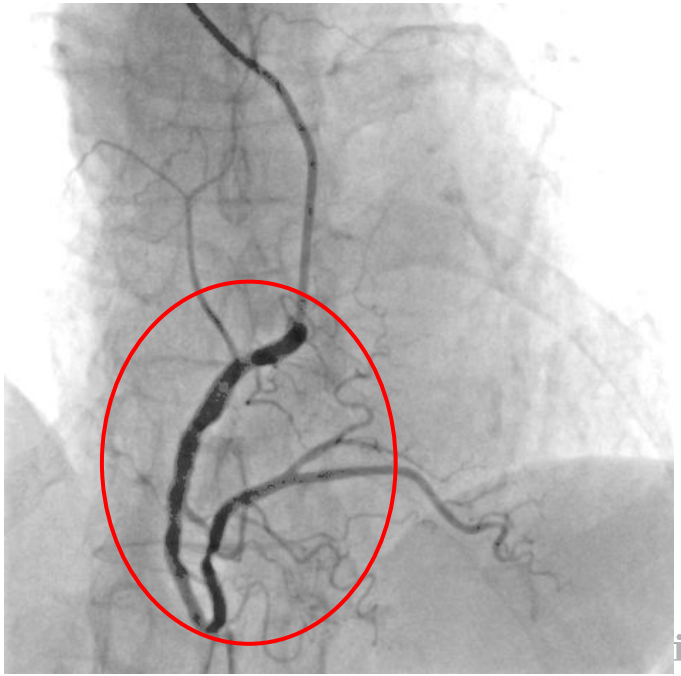
```
# brighten the image
brightened = cv.convertScaleAbs(grayscale_image, alpha=1.3, beta=20)
# save the brightened image
cv.imwrite(f'{angiogram}_2_brightened.png', brightened)


# darken only the dark regions to enhance contrast
darkened_img = brightened.copy()
darkened_img[darkened_img < 100] = darkened_img[darkened_img < 100] * 0.7
darkened_img = darkened_img.astype(np.uint8)
# darkened_img =
cv.imwrite(f'{angiogram}_3_darkened.png', darkened_img)


# Apply pseudocolor
pseudocolored_image = apply_pseudocolor(darkened_img)
# save pseudocolored image
cv.imwrite(f'{angiogram}_4_pseudocolored.png', pseudocolored_image)
```

Pre-processing methods: convertScaleAbs and Darkening

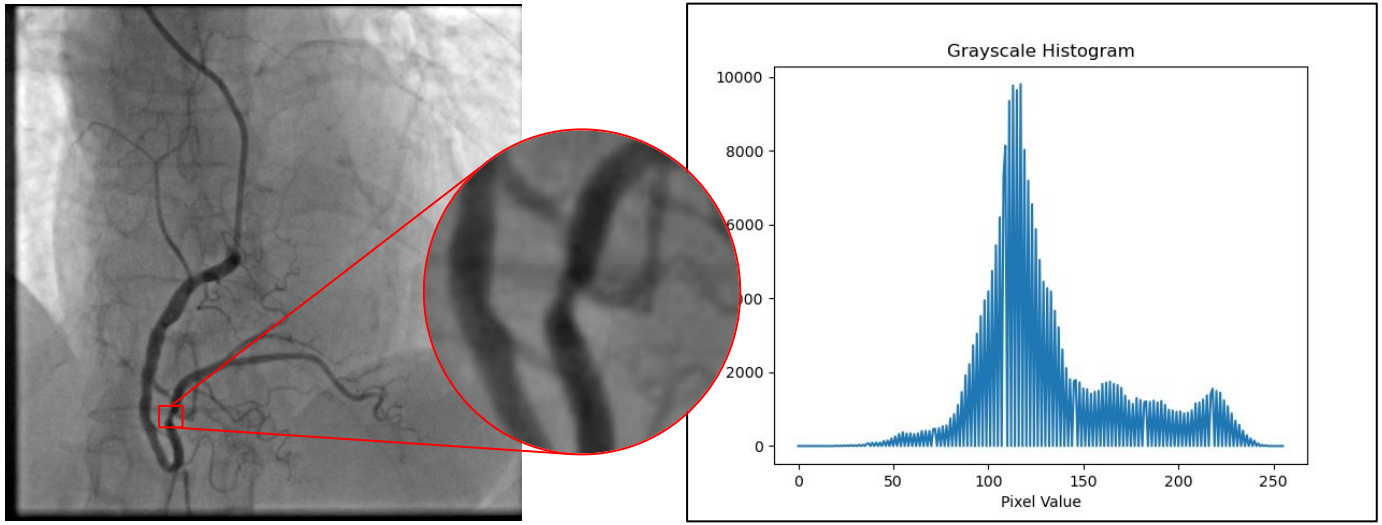
Brightened Image



Darkened Image



Recall this small region is what needs to be detected in the original angiogram image

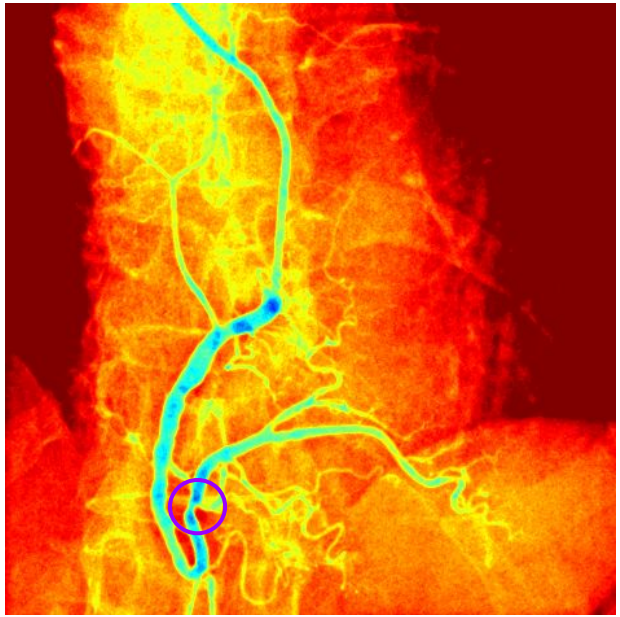


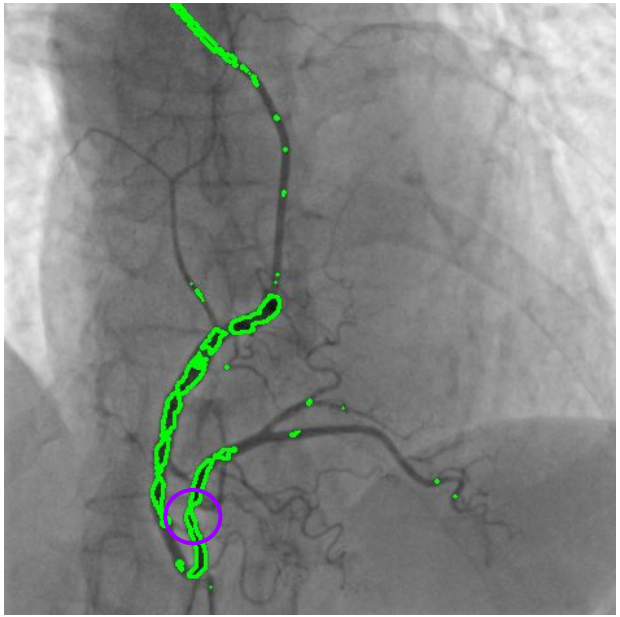
We are trying to detect: 1) thinned region and 2) the subtle coloration change dark → light → dark

Code sample for testing pseudo colorization to contour detection with built in Jet colormap parameter for colormap

```
# PSEUDOCOLOR: Requires 8-bit input
pseudocolored_image = cv.applyColorMap(img_8bit, cv.COLORMAP_JET)

cv.imwrite(os.path.join(save_folder, f"{base_name}_pseudocolored.png"), pseudocolored_image)

cv.imshow("Pseudocolored Image", pseudocolored_image)
cv.waitKey(0)

# MASKING: Detect blue transition regions
lower_blue = (200, 150, 0)   # Adjust these based on JET colormap output to get improbved results (manual tuning per image)
upper_blue = (255, 255, 150)
mask = cv.inRange(pseudocolored_image, lower_blue, upper_blue)

cv.imshow("Mask Image", mask)
cv.waitKey(0)
cv.imwrite(os.path.join(save_folder, f"{base_name}_mask.png"), mask)

# CONTOURS
contours, _ = cv.findContours(mask, cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE)

# Filter contours by area to remove noise and irrelevant details
valid_contours = [cnt for cnt in contours if 500 < cv.contourArea(cnt) < 1000]

# DRAWING: Create a BGR version of the original scan to draw green lines on
# We cannot draw color lines on a grayscale image.
final_output_img = cv.cvtColor(img_8bit, cv.COLOR_GRAY2BGR)

# Draw green contours (0, 255, 0)
cv.drawContours(image=final_output_img, contours=valid_contours, contourIdx=-1, color=(0, 255, 0), thickness=2)
```

Pseudo-colorization and contour detection to isolate disease areas





From this and the following slide we see how this simplistic method alone is not effective at capturing diseased regions of the angiogram

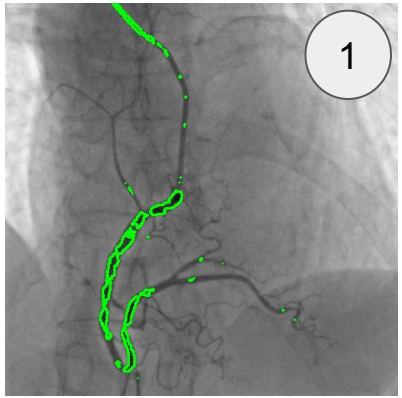
---

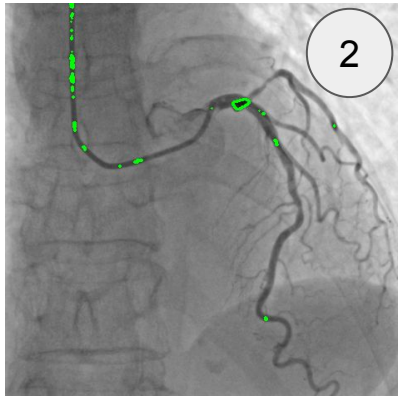
- **Variation within the vessels flow for images we very similar histograms**
  - Causes improper contour detection for transitions where we end up just detecting all possible transition regions rather than the important ones

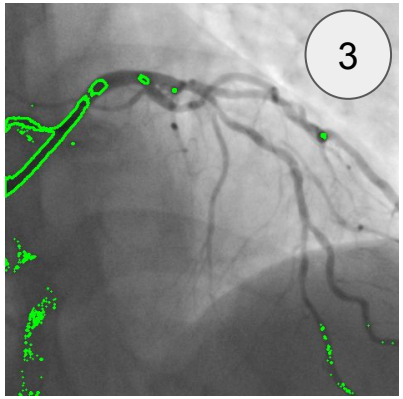
- **Completely different histogram show that images can result in lots of speckle noise after contour detection/thresholding**
  - Each image would need its own tuned parameters
  - Yes we could try unique thresholding for all images → Doing manually for 300 images in the dataset or for any new angiogram would defeat the purpose of an easy access tool

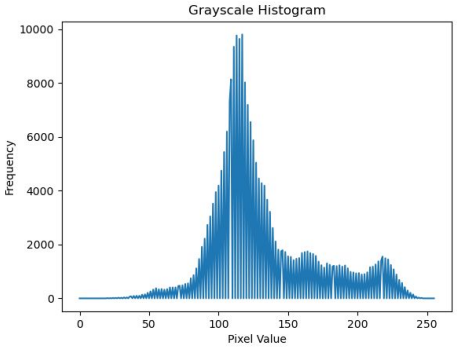
- **What this tells us is that our new method needs to capture more information about the vessel sizes themselves specifically information on detecting the change in vessel width/diameter → skeletonization**

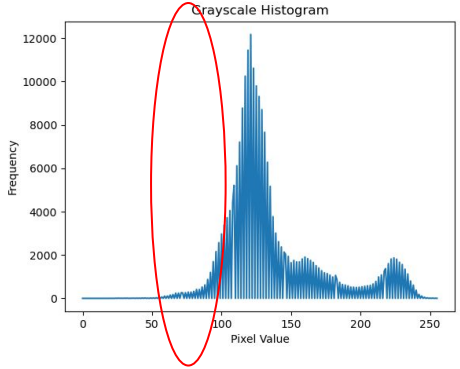
Large variation among angiograms

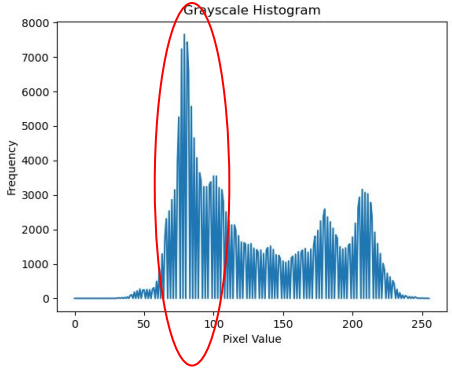












Method 1 Notes/Details:

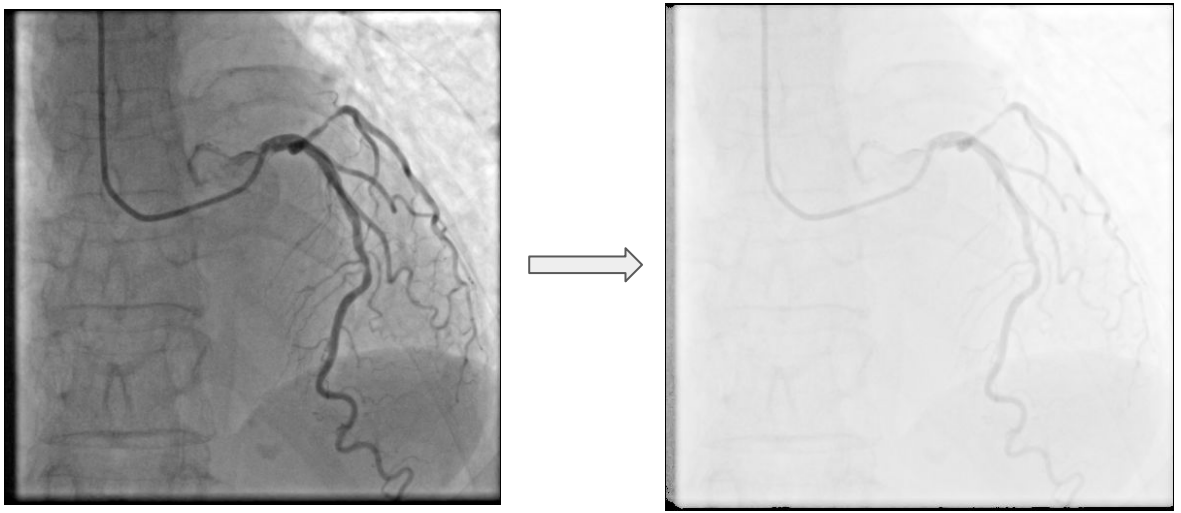
As was mentioned in our class presentation these images that were provided to us in this dataset were unlabelled as to if they were 8 bit images or 10 bit images. Most medical images by standard are 10 bit images and after some deeper digging it was found that the images are 10 bit. However, our methods didn't account for this as we just directly read the image into our OpenCV pipeline.

Looking into the read openCV function this will automatically cast down the image to an 8 bit grayscale if we don't handle it properly and as such we lost some of the grayscale image quality when processing the 8 bit version of the angiograms.

However the pipelines presented show promise still given this small error and given more time to adjust the pipeline and preprocessing for the images, a better more accurate result could be achieved with the more detail levels of the 10 bit grayscale image.

Pre-processing: binarization

- Before using skeletonization, need to create a binary image
- Problem: lot of uneven shadows in the background
- First step: gamma correction used to adaptively brighten



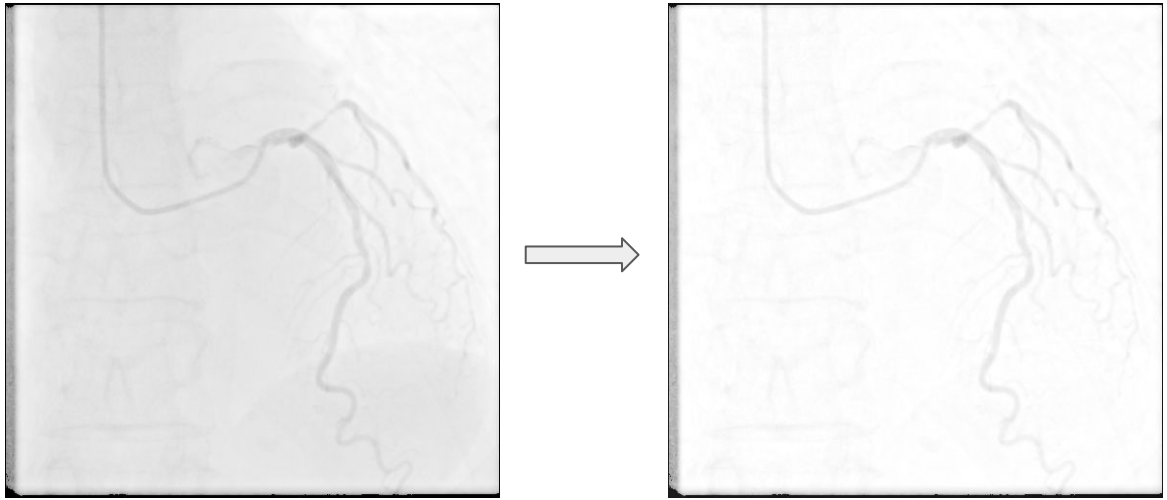
Gamma Correction

```
def gamma_adjust(x):
    global gamma
    gamma = cv2.getTrackbarPos('gamma', window) / 100
    img_np = np.array(img, dtype=np.float32)
    brightened = 255 * (img_np / 255) ** gamma
    brightened = brightened.astype(np.uint8)

    global gray
    gray = cv2.cvtColor(brightened, cv2.COLOR_BGR2GRAY)
    cv2.imshow(window, brightened)

#creating each slider
cv2.createTrackbar('gamma', window, 0, 100, gamma_adjust)
```

- Created trackbar for user to easily change gamma value for each image
- One value doesn't work for all images (e.g darker angiograms vs lighter ones)

Pre-processing: binarization

- Problem: arteries and background have same color
- 2nd step: blackhat operation to extract dark objects (arteries) on light background



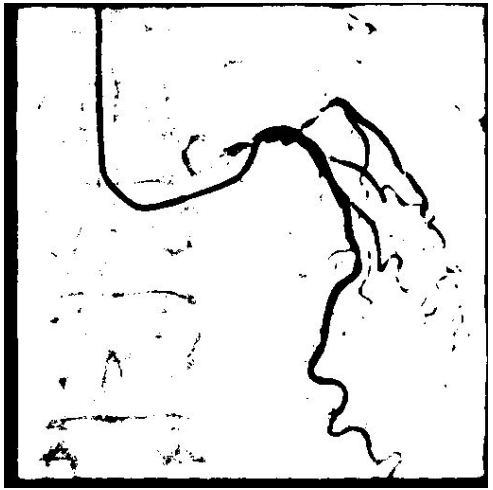
Blackhat Operation

```
def blackhat_adjust(x):
    global kernel_size
    kernel_size = cv2.getTrackbarPos('kernel_size', window)

    if (kernel_size % 2) == 0: kernel_size += 1

    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (kernel_size, kernel_size))
    global gray_blackhat
    gray_blackhat = cv2.morphologyEx(gray, cv2.MORPH_BLACKHAT, kernel)
    gray_blackhat = cv2.bitwise_not(gray_blackhat)

    cv2.imshow(window, gray_blackhat)

cv2.createTrackbar('kernel_size', window, 3, 99, blackhat_adjust)
```

- Experimented with kernel size
  - One kernel size doesn't work well for all images
- Created trackbar for user to easily change kernel size for each image
- Kernel sizes from 3 to 30 all impact blackhat operation's effectiveness

Binarization Results

- 3rd step: binary threshold to convert to black & white



Binary Threshold

```
def bin_adjust(x):
    global thresh
    thresh = cv2.getTrackbarPos('thresh', window)

    global bin
    ret, bin = cv2.threshold(gray_blackhat, thresh, 255, cv2.THRESH_BINARY)

    cv2.imshow(window, bin)

cv2.createTrackbar('thresh', window, 0, 255, bin_adjust)
```

- Created trackbar for user to easily change threshold value for each image
- Higher threshold (245+) usually worked best

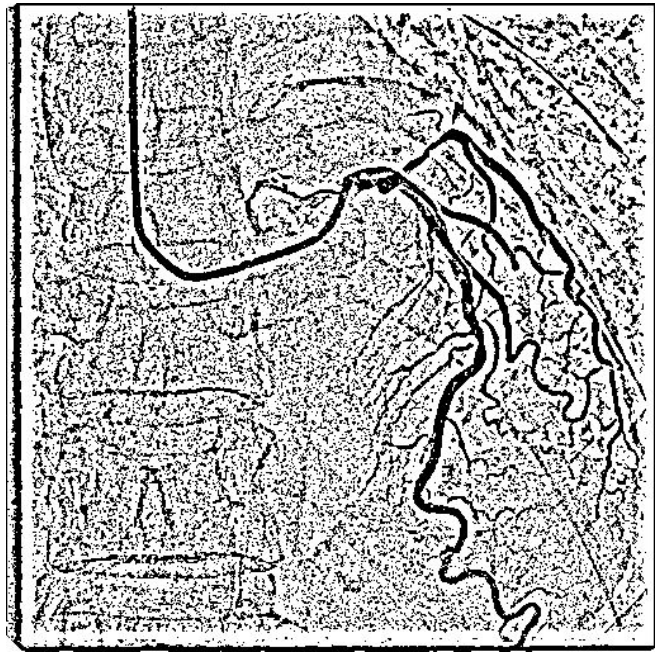
Methods that didn’t work

These methods were experimented with but they left a lot of noise in the background:

- Adaptive mean & gaussian thresholding
- Edge enhancement with Gaussian Blur and cv2.addWeighted
- cv2.convertScaleAbs with varied values for alpha and beta
- cv2.createCLAHE → was more promising than the other methods, could work if optimal parameters are found
- Closing & opening operations → removes a lot of background noise but also detail from the arteries

Methods that didn't work

GaussianBlur + cv2.addWeighted + binary threshold



GaussianBlur + cv2.addWeighted + adaptiveThreshold

Diameter Stenosis

DS stands for "Diameter Stenosis" - standard medical term used in cardiovascular imaging.

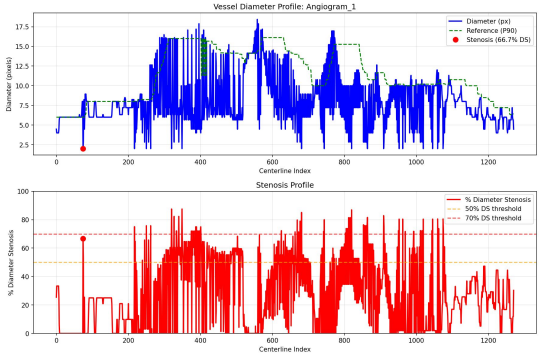
%DS (Percent Diameter Stenosis) is a clinical measurement that quantifies how much a blood vessel has narrowed compared to its normal reference diameter.

Formula:

%DS = (1 - (Stenotic Diameter / Reference Diameter)) × 100%

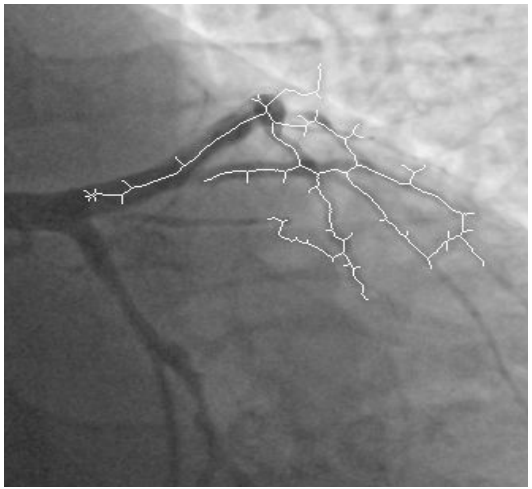
Clinical Interpretation:

%DS Range	Severity	Clinical Action
0-30%	Minimal	Monitoring
30-50%	Mild	Lifestyle changes
50-70%	Moderate	Medical therapy
70-90%	Severe	Intervention needed
>90%	Critical	Emergency intervention



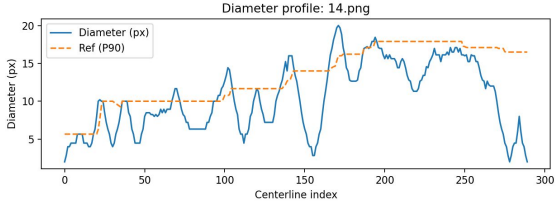
Method 2: Detecting narrowed arteries using skeletonization

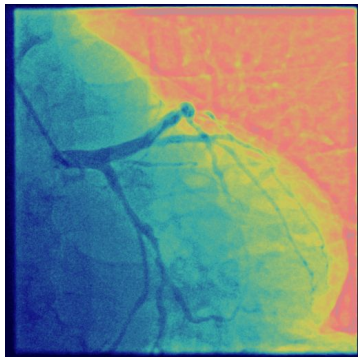
Skeletonization allows us to measure vessel diameter along a 1-pixel centerline and automatically detect narrowed regions.

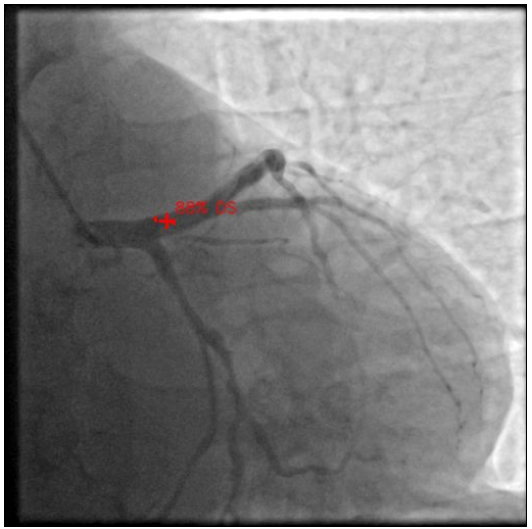


Method 2: Detecting narrowed arteries using skeletonization, DV% and Intensity

Measuring diameter and intensity along the centerline allows us to automatically detect and localize stenosis.







How our stenosis detector works

- A point is flagged if it has **high %DS** and a significant **intensity dip** (reduced contrast flow).
- We summarise each image with:
  a. **Max %DS**
  b. **Count of points in 80–90% and ≥90% stenosis range**
- Output: overlay image + CSV + master summary.csv for quick review across hundreds of angiograms.

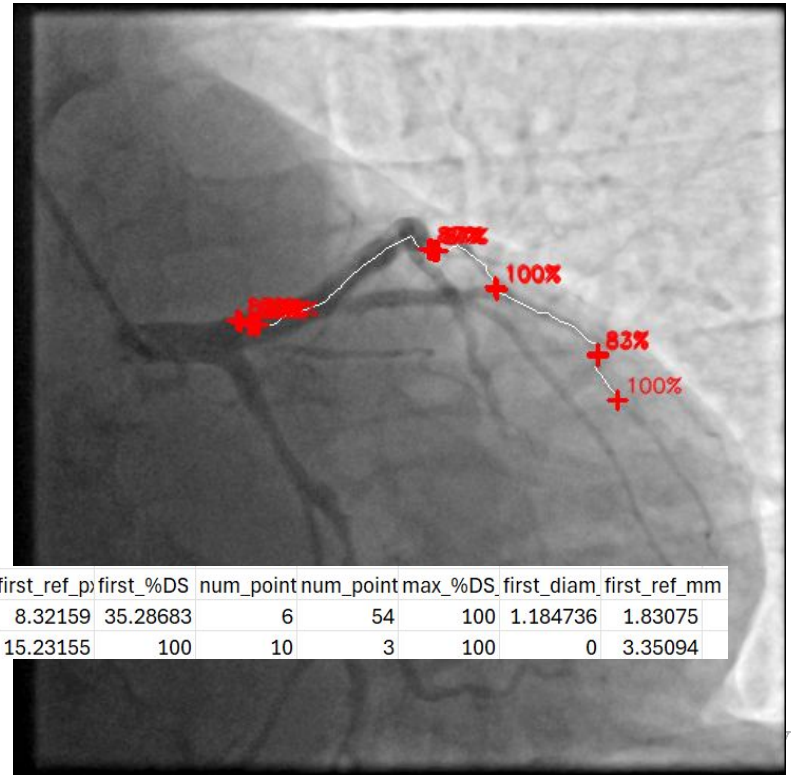
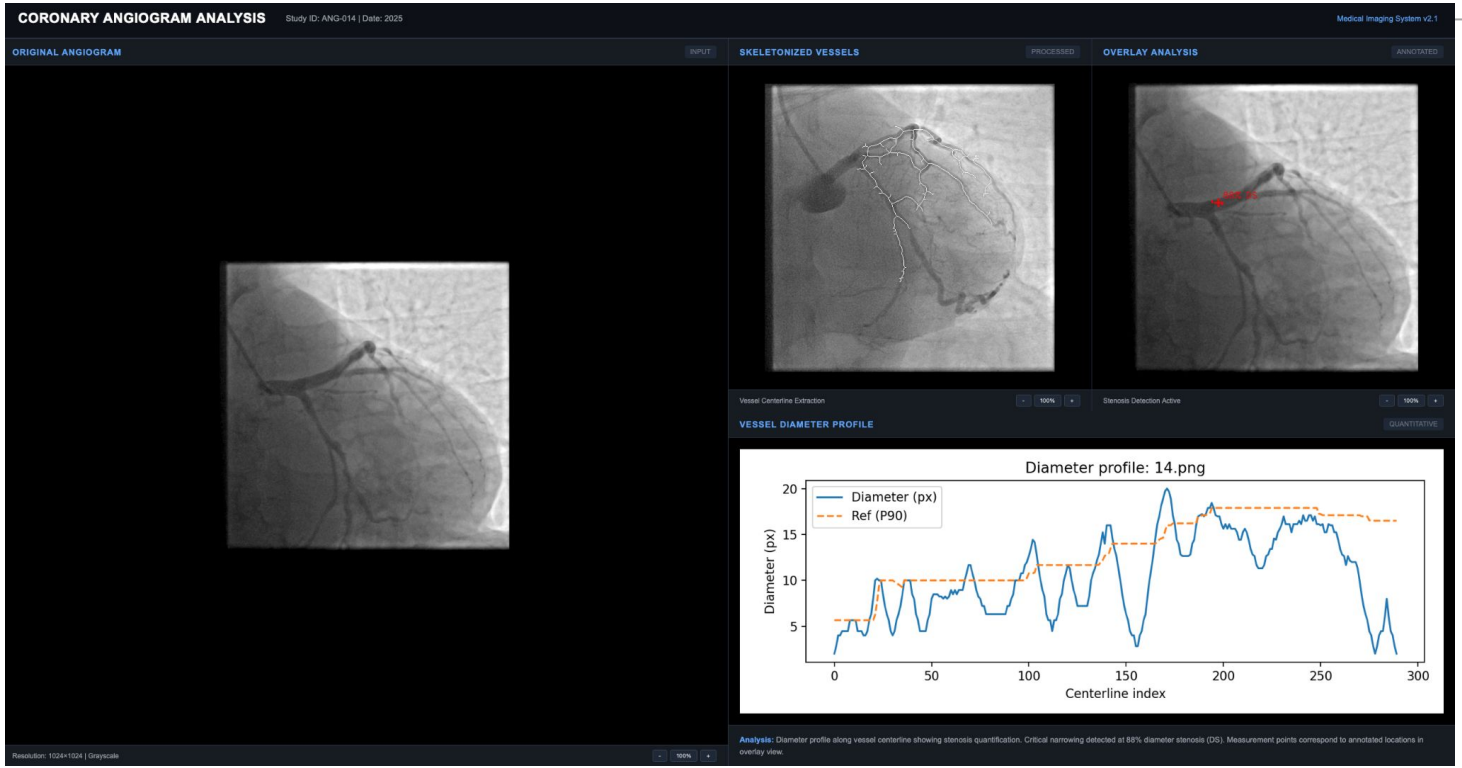


image	overlay_pa	per_image	detections	num_dete	first_x	first_y	first_diam_	first_ref_p	first_%DS	num_point	num_point	max_%DS_	first_diam_	first_ref_mm
D:\Work\C	D:\Work\C	D:\Work\C	D:\Work\C	10	25	154	5.385165	8.32159	35.28683	6	54	100	1.184736	1.83075
D:\Work\C	D:\Work\C	D:\Work\C	D:\Work\C	14	159	212	0	15.23155	100	10	3	100	0	3.35094

Results Viewer for doctors: Test GUI



Future improvements & scaling

- Techniques like stereovision to create 3D rendering of heart vessels (angiograms come in pairs of 2D images)
- Establishing Reference Vessel Diameter (RVD) and Minimal Lumen Diameter (MLD) in consideration of classification and trying ML methods for detecting diseased regions color transitions
- Creating a more detailed Graphical User Interface (GUI) for adaptive and interactive adjustment of Angiogram processing on the fly

Funding Request

$150k

- Access to large angiogram database
- Integration of LLMs for increased accuracy
- Institutional Review Board approval
- Development of an interactive GUI for doctors

Dataset

The ARCADE dataset (Automatic Region-based Coronary Artery Disease Diagnostics using X-ray Angiography) provides a large-scale, expert-annotated resource for developing and benchmarking AI models in coronary artery disease (CAD) diagnostics. It includes:

Annotations created and cross-validated by experienced cardiologists using the CVAT tool

Images sourced from real-world patients (age 19–90) at the Research Institute of Cardiology and Internal Diseases (Almaty, Kazakhstan)

Images obtained using Philips Azurion 3 and Siemens Artis Zee angiographs

Frame resolution: 512 × 512 pixels

This dataset supports research in medical image analysis, deep learning, quantitative coronary analysis, and automated diagnostics. It was first introduced as part of the ARCADE Challenge at MICCAI 2023.

https://www.kaggle.com/datasets/nirmalgaud/arcade-dataset

Github

All the code files relevant to the project : https://github.com/Sangramxd/F25_CVE_Denver.git

The complete implementation of our coronary vessel analysis pipeline is available in the linked GitHub repository. It includes all modules for preprocessing, Frangi-based vessel segmentation, skeleton extraction, diameter measurement, and %DS stenosis quantification, along with batch-processing scripts and example outputs. The repository provides full transparency of our methods and enables easy replication or extension of the results presented in this project.

Carnegie Mellon University

Thank you for listening!

Any questions?